

机器视觉-实践项目 4

国嘉通^{*}

2025 年 10 月 7 日

摘要

机器视觉第四次实验课程主要包含两大部分的练习，一是图像形态学处理，二是边缘检测处理。对于图像形态学处理部分，分别从腐蚀、膨胀、开运算、闭运算、骨架提取五个方面进行详细阐释。对于边缘检测处理部分，分别从 Sobel 算子与 Canny 算子两个角度进行阐释。另外，还介绍了 K-means 与 Mean Shift 两种图像分割方法。希望本篇文章是一篇内容翔实、方便随时翻阅复习学习 OpenCV 的学习笔记。

Now the CODE is available at:

<https://github.com/GplasT0810/Machine-Vision-Experiment.git>

长沙理工大学

卓越工程师学院 绿色智慧交通 2401 班

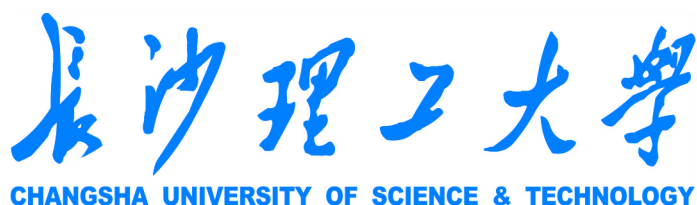
202404080608

国嘉通

^{*}Tel.+86-166 8130 6085, E-mail:guojiatom2006@outlook.com

目录

1	形态学处理	3
1.1	腐蚀运算 (Erosion)	3
1.2	膨胀运算 (Dilation)	4
1.3	开运算 (Opening)	6
1.4	闭运算 (Closing)	7
1.5	骨架提取 (Skeletonization)	9
2	边缘检测操作	10
3	图像分割	12



Completed Time: Oct.1st.2025

Guo Jiatong , Green & Intelligent Transportation 2401
Elite Engineering School , Changsha University of Science & Technology.
Changsha 410114 , Hunan , China

1 形态学处理

本部分分别从腐蚀、膨胀、开运算、闭运算、骨架提取五个方面进行详细阐释。

1.1 腐蚀运算 (Erosion)

- 原理

使用结构元素逐个像素地扫描图像，仅当结构元素完全覆盖前景区域时，图像的中心像素才会被保留。若设图像集合为 A ，结构元素为 B ，则腐蚀运算定义为

$$A \ominus B = \{z \mid B_z \subseteq A\}$$

即结构元素 B 在图像 A 上滑动，只有当 B 完全包含于 A 时，中心像素才会被保留。

- 作用

去除小噪点（噪点需要小于结构元素）、分离粘连物体并进行边缘提取前的预处理、缩小前景区域等。

- 代码实现

```
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
eroded = cv2.erode(binary, kernel, iterations=1)
```

```
# 图像腐蚀运算
import cv2
import numpy as np
import matplotlib.pyplot as plt

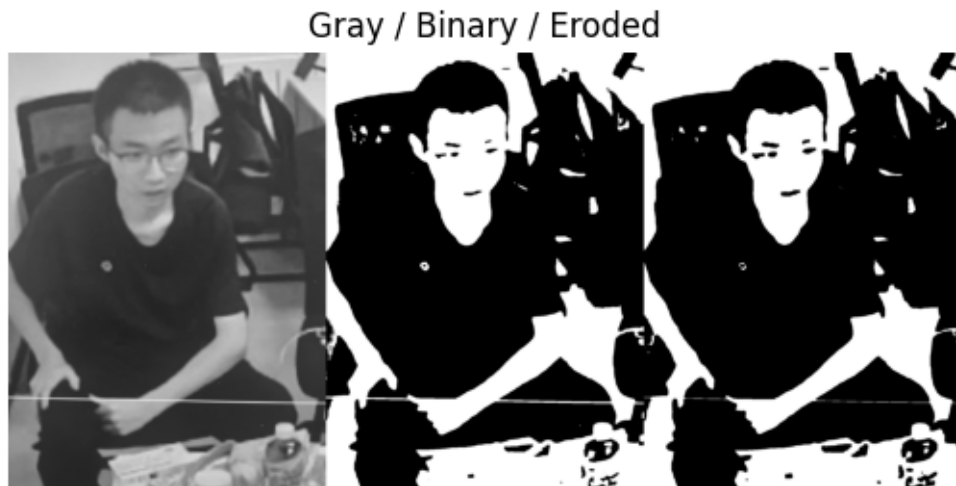
img_path = '/Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_
↳Experiment 4/zb.jpg'
img = cv2.imread(img_path)
if img is None:
    raise FileNotFoundError('图片路径错误或文件不存在')

gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# 定义腐蚀核
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
eroded = cv2.erode(binary, kernel, iterations=1)
```

```
vis = np.hstack([gray, binary, eroded])
vis = cv2.cvtColor(vis, cv2.COLOR_BGR2RGB)
plt.imshow(vis)
plt.axis('off')
plt.title('Gray / Binary / Eroded')
```

```
Text(0.5, 1.0, 'Gray / Binary / Eroded')
```



1.2 膨胀运算 (Dilation)

- 原理
结构元素覆盖任意一个前景像素（结构元素与前景有交集），则中心即设为前景。若设图像集合为 A ，结构元素为 B ，则膨胀运算定义为

$$A \oplus B = \{z \mid B_z \cap A \neq \emptyset\}$$

- 作用
填补小孔洞，连接断裂区域、扩大前景区域等，还可以提取图像边缘，膨胀运算是腐蚀运算的对偶操作。
- 代码实现

```
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
dilated = cv2.dilate(binary, kernel, iterations=1)
```

```
# 图像膨胀运算
import cv2
import numpy as np
```

```

import matplotlib.pyplot as plt

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_
↳Experiment 4/zb.jpg'
img = cv2.imread(img_path)
if img is None:
    raise FileNotFoundError('图片路径错误或文件不存在')

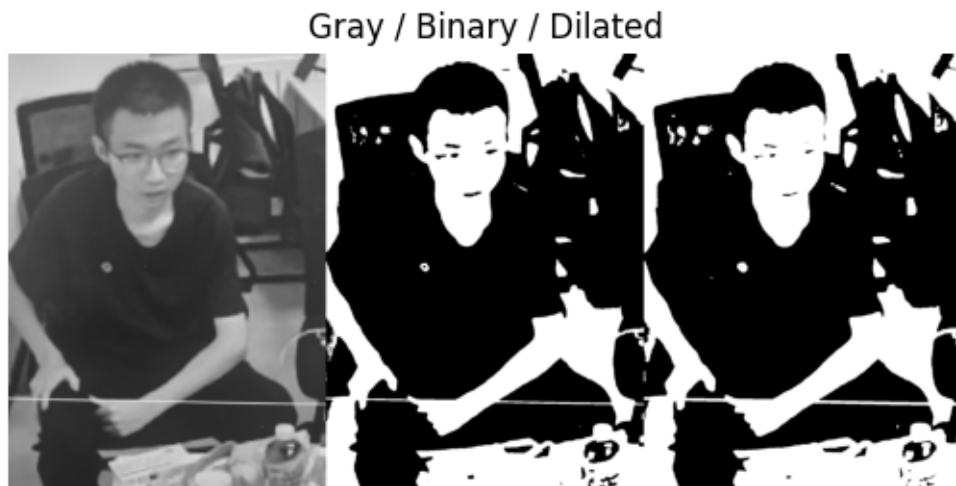
# 图像灰度处理与二值化
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)
# 前景为黑色->THRESH_BINARY
# 前景为白色->THRESH_BINARY_INV

# 定义膨胀核
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
dilated = cv2.dilate(binary, kernel, iterations=1)

vis = np.hstack([gray, binary, dilated])
vis = cv2.cvtColor(vis, cv2.COLOR_BGR2RGB)
plt.imshow(vis)
plt.axis('off')
plt.title('Gray / Binary / Dilated')

```

```
Text(0.5, 1.0, 'Gray / Binary / Dilated')
```



1.3 开运算 (Opening)

- 定义

开运算本质上是腐蚀运算与膨胀运算的叠加，即先进行腐蚀运算，再进行膨胀运算。若设图像集合为 A ，结构元素为 B ，则膨胀运算定义为

$$\text{Opening}(A) = (A \ominus B) \oplus B$$

- 作用

其可以去除小物体，同时保留大物体形状不变，还可以用于平滑轮廓，断开狭窄连接，此外，它能够抑制亮噪声。在实际应用中，适用于文档图像去噪、图像分割的预处理（清理噪点）、提取大结构忽略局部细节等等场景。

- 代码实现

```
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel)
```

```
# 图像开运算
import cv2
import numpy as np
import matplotlib.pyplot as plt

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_
↳Experiment 4/zb.jpg'
img = cv2.imread(img_path)
if img is None:
    raise FileNotFoundError('图片路径错误或文件不存在')

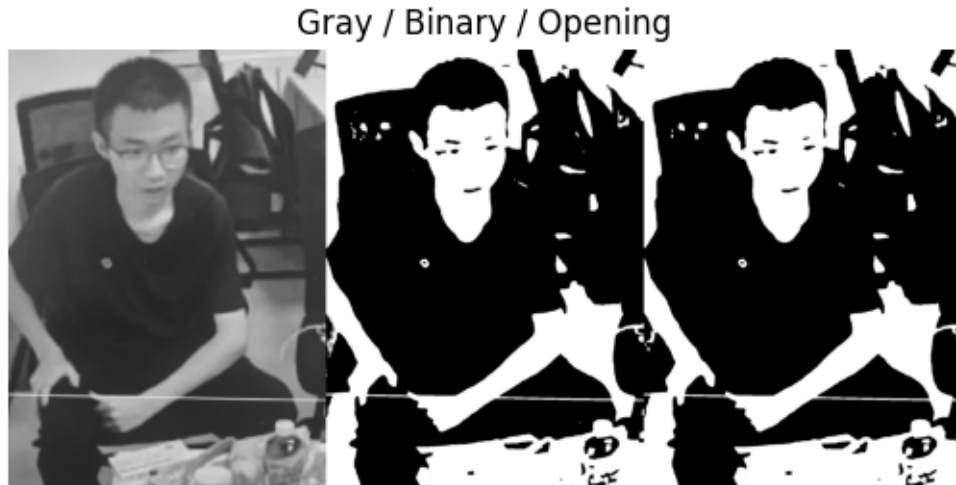
# 灰度变换与二值化
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# 定义结构元
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel, iterations=1)

vis = np.hstack([gray, binary, opening])
vis = cv2.cvtColor(vis, cv2.COLOR_BGR2RGB)
```

```
plt.imshow(vis)
plt.title('Gray / Binary / Opening')
plt.axis('off')
```

```
(np.float64(-0.5), np.float64(3071.5), np.float64(1407.5), np.float64(-0.5))
```



1.4 闭运算 (Closing)

- 定义

闭运算本质上也是腐蚀运算与膨胀运算的叠加，只不过其顺序恰好与开运算相反。其是先进行膨胀运算，再进行腐蚀运算。若设图像集合为 A ，结构元素为 B ，则膨胀运算定义为

$$\text{Closing}(A) = (A \oplus B) \ominus B$$

- 作用

填补前景内部的小空洞，连接断裂部分，平滑边界的同时能消除暗噪声，其有保持物体面积基本不变的优良特性。因此其适合用于连接断裂的字符或线条以及在医学图像处理领域中的组织区域修复等。

- 代码实现

```
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
closing = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel)
```

```
# 图像闭运算
import cv2
import numpy as np
import matplotlib.pyplot as plt
```

```

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_
↳Experiment 4/zb.jpg'
img = cv2.imread(img_path)
if img is None:
    raise FileNotFoundError('图片路径错误或文件不存在')

# 灰度变换与二值化
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU)

# 定义结构元
kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (5, 5))
closing = cv2.morphologyEx(binary, cv2.MORPH_CLOSE, kernel, iterations=1)

vis = np.hstack([gray, binary, closing])
vis = cv2.cvtColor(vis, cv2.COLOR_BGR2RGB)
plt.imshow(vis)
plt.title('Gray / Binary / Closing')
plt.axis('off')

```

```

(np.float64(-0.5), np.float64(3071.5), np.float64(1407.5), np.float64(-0.5))

```



1.5 骨架提取 (Skeletonization)

- 原理

骨架是前景区域的“中轴线”，骨架提取操作能将前景区域细化为单像素宽的“骨架”，同时保持其拓扑结构不变，thinning 至单像素宽。具体算法的实现基于 Zhang-Suen 快速并行细化算法、Guo-Hall 算法以及基于距离变换的方法等。

- 应用

由于其具有骨架单像素宽、能够很好地保持图像的连通性等优良特性，其经常会被用于字符识别、形状分析、路径提取与规划等领域。具体来说，在手写数字或字母的识别，血管、道路裂纹的提取以及图像压缩与结构分析等领域均有应用。

- 代码实现

```
from skimage.morphology import skeletonize
skeleton = skeletonize(binary_bool)
```

```
# 骨架操作
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.morphology import skeletonize
from skimage.util import img_as_bool
#OpenCV 没有单步骨架 API，采用 Zhang-Suen 快速并行细化算法，安装 scikit-image

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_
↳Experiment 4/zb.jpg'
img = cv2.imread(img_path)
if img is None:
    raise FileNotFoundError('图片路径错误或文件不存在')

# 灰度转换与二值化
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
_, binary = cv2.threshold(gray, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU)

binary_bool = img_as_bool(binary // 255)
skeleton = skeletonize(binary_bool)

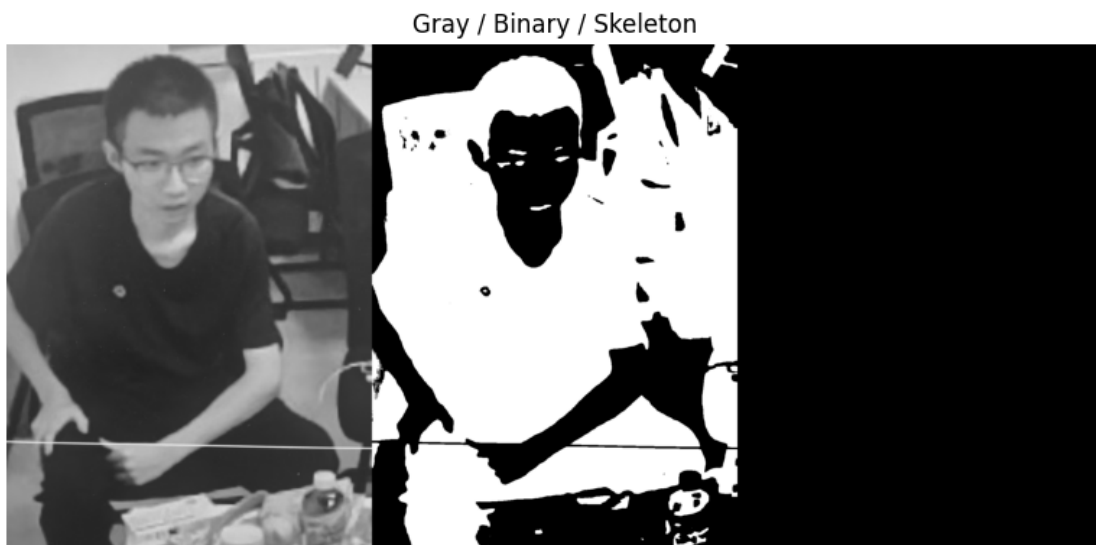
skeleton_uint8 = skeleton.astype(np.uint8) * 255
```

```

kernel = cv2.getStructuringElement(cv2.MORPH_CROSS, (3,3))
skeleton_thick = cv2.dilate(skeleton_uint8, kernel, iterations=1)

vis = np.hstack([gray, binary, skeleton_thick])
plt.figure(figsize=(12,4))
plt.imshow(vis, cmap='gray')
plt.title('Gray / Binary / Skeleton')
plt.axis('off')
plt.tight_layout()
plt.show()

```



2 边缘检测操作

本章节分为基于 Sobel 算子、Canny 算子的边缘检测操作。

Sobel 算子

- 原理

Sobel 算子基于一阶导数的近似，分别计算水平（ x 方向）与垂直（ y 方向）梯度，并合成梯度幅值：

$$G_x = \begin{bmatrix} -1 & 0 & 1 \\ -2 & 0 & 2 \\ -1 & 0 & 1 \end{bmatrix}, \quad G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ 1 & 2 & 1 \end{bmatrix}, \quad |G| = \sqrt{G_x^2 + G_y^2}$$

- 特点

其具有简单、快速高效的优点，不过其对噪声敏感，且边缘较粗，定位精准性一般，可用于初步的边缘提取或搭配 Canny 算法共同使用（作为 Canny 算法的子步骤）。

- 代码实现

```
sobelx = cv2.Sobel(gray, cv2.CV_16S, 1, 0, ksize=3)
sobely = cv2.Sobel(gray, cv2.CV_16S, 0, 1, ksize=3)
sobel = cv2.magnitude(sobelx.astype(np.float32), sobely.astype(np.float32))
sobel = cv2.convertScaleAbs(sobel)
```

Canny 算子

- 步骤

高斯滤波去噪 → 计算梯度幅值与方向 → 非极大值抑制（细化边缘）→ 双阈值检测 → 边缘连接（滞后阈值）

- 特点

边缘细、连续、定位准确且抗噪能力强，但是对参数敏感。建议低阈值为高阈值的 $\frac{1}{2} - \frac{1}{3}$ 。基于以上特点，易于得知 Canny 算法比 Sobel 算法更具优势（或者说更具应用前景），其可以被用于目标检测、轮廓提取等，还可以用于图像分割的预处理以及工业缺陷的检测等多个领域。

- 代码实现

```
canny = cv2.Canny(gray, 50, 150)
```

```
# 边缘检测
import cv2
import numpy as np
import matplotlib.pyplot as plt

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_
↳Experiment 4/zb.jpg'
img = cv2.imread(img_path)
if img is None:
    raise FileNotFoundError('图片路径错误')
gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)

#Sobel 算子
sobelx = cv2.Sobel(gray, cv2.CV_16S, 1, 0, ksize=3) # 水平梯度
sobely = cv2.Sobel(gray, cv2.CV_16S, 0, 1, ksize=3) # 垂直梯度
```

```

sobel_f = cv2.magnitude(sobelx.astype(np.float32), sobely.astype(np.float32)) #
合成梯度幅值
sobel = cv2.convertScaleAbs(sobel_f) # 转 8 位

#Canny 算子
canny = cv2.Canny(gray, 10, 100)

vis = np.hstack([gray, sobel, canny])
vis = cv2.cvtColor(vis, cv2.COLOR_BGR2RGB)
plt.imshow(vis)
plt.title('Gray / Sobel / Canny')
plt.axis('off')

```

```

(np.float64(-0.5), np.float64(3071.5), np.float64(1407.5), np.float64(-0.5))

```

Gray / Sobel / Canny



3 图像分割

K-means 聚类分割

- 原理

将像素颜色视为三维向量 (RGB)，在颜色空间中进行 K-means 聚为 K 类，最小化类内平方误差并形成区域。其目标函数：

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

- 特点

简单快速，易于实现，且无需额外标注，可用于任意维特征，但需预设 K 值，且对噪声敏感，易忽略空间关系，形成“椒盐”碎片。

- 代码实现

```
kmeans = KMeans(n_clusters=4, random_state=0).fit(data)
labels = kmeans.labels_.reshape(h, w)
```

Mean Shift 分割

- 原理

基于核密度估计，密度梯度上升，自动迭代寻找局部密度最大值（模态），将像素归属到最近模态，形成自然区域。核函数：

$$K(x) = \exp\left(-\frac{\|x\|^2}{2\sigma^2}\right)$$

- 特点

无需预设聚类数 K，边界自然且平滑，能发现任意形状区域，但计算量大，高维特征下收敛速度慢，需超像素加速，且带宽选择敏感。

- 代码实现

```
segments = slic(img_rgb, n_segments=500, compactness=10)
centers = np.array([data[segments == i].mean(axis=0) for i in np.unique(segments)
])
labels = pairwise_distances_argmin(data, centers).reshape(h, w)
```

```
# 附加
import cv2, numpy as np, time
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
from sklearn.metrics import pairwise_distances_argmin
from skimage.segmentation import slic, mark_boundaries
from skimage.filters import sobel

img_path = '/Users/guo2006/myenv/Machine Vision Experiment/Machine Vision_
↳Experiment 4/zxc.jpg'
img = cv2.imread(img_path)
if img is None:
    raise FileNotFoundError('路径错误')
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
```

```

h, w, c = img_rgb.shape
data = img_rgb.reshape(-1, 3)

#K-means
t0 = time.time()
kmeans = KMeans(n_clusters=4, random_state=0, n_init='auto').fit(data)
kmeans_label = kmeans.labels_.reshape(h, w)
kmeans_seg = kmeans.cluster_centers_[kmeans_label].astype(np.uint8)
tk = time.time() - t0

#Mean-Shift
t0 = time.time()
# 先用 SLIC 超像素加速 Mean-Shift, 否则 2K×1K 图会跑不动
segments = slic(img_rgb, n_segments=500, compactness=10, start_label=0)
seg_centers = np.array([data[segments.reshape(-1) == i].mean(axis=0)
                        for i in np.unique(segments)])
ms_labels = pairwise_distances_argmin(data, seg_centers).reshape(h, w)
mean_shift_seg = seg_centers[ms_labels].astype(np.uint8)
tm = time.time() - t0

# 评价指标函数
def boundary_smoothness(seg_gray):
    """ 沿分割边界的梯度均值, 越小越平滑 """
    edge = sobel(seg_gray)
    return edge.mean()

def region_consistency(seg_lab, label_map):
    """ 各区域内部分方差均值, 越小越一致 """
    var = []
    for l in np.unique(label_map):
        mask = label_map == l
        var.append(seg_lab[mask].var(axis=0).mean())
    return np.mean(var)

def detail_preservation(seg_gray):
    """ 边缘像素占比, 越高细节越多 """

```

```

    edge = sobel(seg_gray)
    return (edge > 0.05).mean()

# 转灰度 + 计算
kmeans_gray = cv2.cvtColor(kmeans_seg, cv2.COLOR_RGB2GRAY)
ms_gray = cv2.cvtColor(mean_shift_seg, cv2.COLOR_RGB2GRAY)

k_smooth = boundary_smoothness(kmeans_gray)
ms_smooth = boundary_smoothness(ms_gray)
k_consist = region_consistency(kmeans_seg.astype(np.float32)/255., kmeans_label)
ms_consist = region_consistency(mean_shift_seg.astype(np.float32)/255.,
    ↪ms_labels)
k_detail = detail_preservation(kmeans_gray)
ms_detail = detail_preservation(ms_gray)

fig, ax = plt.subplots(2, 2, figsize=(10, 8))
ax[0, 0].imshow(img_rgb), ax[0, 0].set_title('Original'), ax[0, 0].axis('off')
ax[0, 1].imshow(mark_boundaries(img_rgb, kmeans_label, color=(1,0,0)))
ax[0, 1].set_title(f'K-means t={tk:.3f}s'), ax[0, 1].axis('off')
ax[1, 0].imshow(mark_boundaries(img_rgb, ms_labels, color=(1,0,0)))
ax[1, 0].set_title(f'Mean-Shift t={tm:.3f}s'), ax[1, 0].axis('off')
ax[1, 1].axis('off')
table = [['Target', 'K-means', 'Mean-Shift'],
          ['Boundary smoothing', f'{k_smooth:.4f}', f'{ms_smooth:.4f}'],
          ['Regional Consistency', f'{k_consist:.4f}', f'{ms_consist:.4f}'],
          ['Details Remain', f'{k_detail:.4f}', f'{ms_detail:.4f}'],
          ['Times', f'{tk:.3f}s', f'{tm:.3f}s']]
ax[1, 1].table(cellText=table[1:], colLabels=table[0], loc='center')
plt.tight_layout()
plt.show()

```

/Users/guo2006/myenv/lib/python3.11/site-packages/sklearn/utils/extmath.py:203:

RuntimeWarning: divide by zero encountered in matmul

```
ret = a @ b
```

/Users/guo2006/myenv/lib/python3.11/site-packages/sklearn/utils/extmath.py:203:

RuntimeWarning: overflow encountered in matmul

```
ret = a @ b
```

```

/Users/guo2006/myenv/lib/python3.11/site-packages/sklearn/utils/extmath.py:203:
RuntimeWarning: invalid value encountered in matmul
    ret = a @ b
/Users/guo2006/myenv/lib/python3.11/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: divide by zero
encountered in matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/guo2006/myenv/lib/python3.11/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: overflow encountered in
matmul
    current_pot = closest_dist_sq @ sample_weight
/Users/guo2006/myenv/lib/python3.11/site-
packages/sklearn/cluster/_kmeans.py:237: RuntimeWarning: invalid value
encountered in matmul
    current_pot = closest_dist_sq @ sample_weight

```

Original



K-means t=0.044s



Mean-Shift t=0.303s



Target	Kmeans	Mean-Shift
Boundary smoothing	0.0089	0.0125
Regional Consistency	0.0000	0.0000
Details Remain	0.0655	0.0684
Times	0.044s	0.303s